

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашняя работа 5

Выполнил:

Смирнов Александр

Группа J3210

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на предоставленный мануал, необходимо было развернуть проект, установить необходимые зависимости и разработать простое SPA для создания и отображения заметок. Приложение должно включать в себя работу с компонентами, роутингом, глобальным состоянием и отправкой сетевых запросов.

Ход работы

1. Инициализация проекта

С помощью команды `npm init vue@latest` был сгенерирован базовый шаблон проекта. В процессе настройки были подключены Vue Router для навигации и Pinia для управления состоянием.

2. Установка зависимостей

Через NPM были добавлены сторонние пакеты, необходимые для работы логики и визуальной части приложения: `npm install axios pinia-persists bootstrap`.

3. Настройка архитектуры

Проект был разбит на логические модули:

- Настроен инстанс `axios` для взаимодействия с REST API
- Создано глобальное хранилище заметок в `src/stores/notes.js`, включающее методы `loadNotes` и `createNote`
- Создан переиспользуемый UI-компонент карточки заметки `NoteCard.vue` и базовый лейаут `BaseLayout.vue`
- Вся логика объединена на главной странице представления `NotesPage.vue`

4. Запуск mock-сервера

Так как для работы приложения требовался бэкенд, локально был развернут фейковый REST API с помощью утилиты `json-server`, куда приложение успешно отправляет POST и GET запросы.

Результат работы интерфейса:

При запуске приложения (npm start) отображается форма добавления новой заметки (рис. 1). Двухнаправленное связывание данных реализовано через директиву v-model.

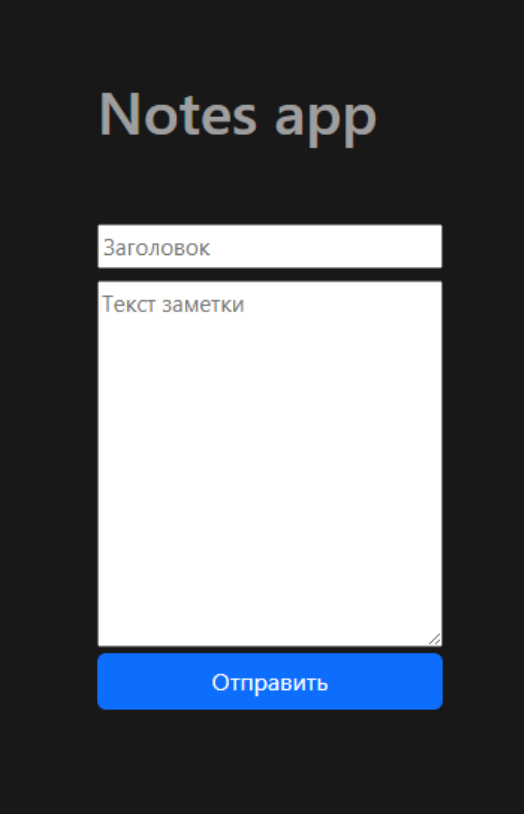
The image shows a mobile application interface titled "Notes app" in a light blue font on a dark blue background. Below the title is a form with two input fields: "Заголовок" (Title) and "Текст заметки" (Note text). The "Текст заметки" field is a larger text area. At the bottom of the form is a red button labeled "Отправить" (Send).

Рис 1. Исходное состояние приложения

После заполнения полей "Заголовок" и "Текст заметки" и нажатия кнопки "Отправить", данные передаются в глобальный стейт и отправляются на сервер. Сразу после этого реактивно обновляется список, и компонент NoteCard отрисовывает добавленную заметку на экране (рис. 2).

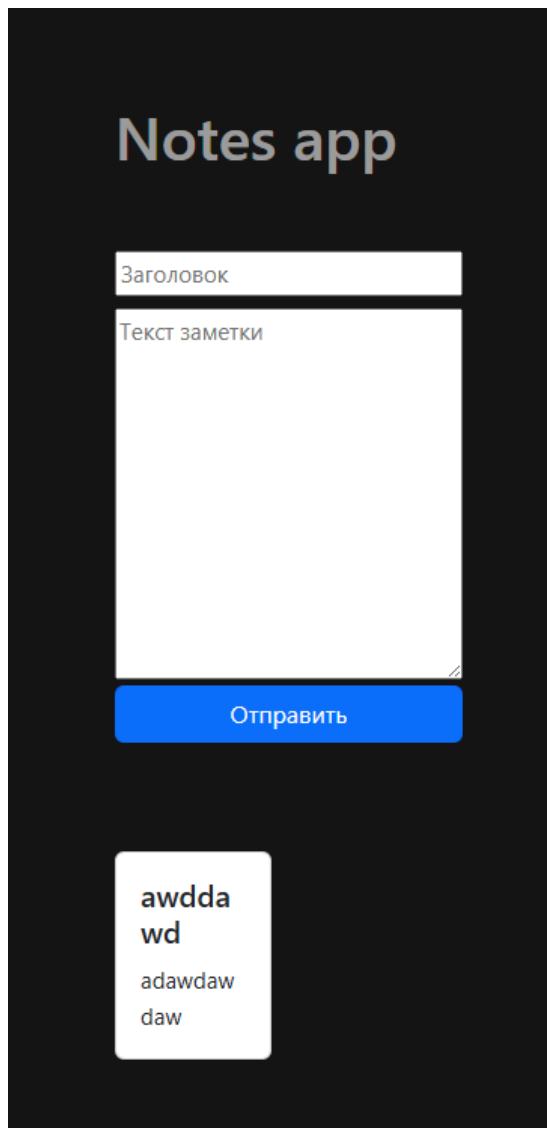


Рис. 2. Успешное добавление и рендеринг новой карточки заметки:

3. Листинг ключевых файлов, отвечающих за работу приложения

src/views/NotesPage.vue (Главное представление):

```
<template>
  <base-layout>
    <h1>Notes app</h1>
    <form ref="noteForm" @submit.prevent="createCard"
class="d-flex flex-column my-5">
      <input type="text" v-model="form.name" class="my-1"
placeholder="Заголовок">
      <textarea cols="30" rows="10" v-model="form.text">
```

```

class="my-1" placeholder="Текст заметки"></textarea>
      <button type="submit" class="btn
btn-primary">Отправить</button>
    </form>

    <div class="row row-cols-1 row-cols-md-2 g-4 mt-5"
id="notes">
      <div class="col" v-for="note in notes" :key="note.id">
        <note-card :name="note.name" :text="note.text" />
      </div>
    </div>
  </base-layout>
</template>

<script>
import BaseLayout from '@/layouts/BaseLayout.vue'
import NoteCard from '@/components/NoteCard.vue'
import { mapActions, mapState } from 'pinia'
import useNotesStore from '@/stores/notes'

export default {
  name: 'NotesPage',
  components: { BaseLayout, NoteCard },
  data() {
    return {
      form: {
        name: '',
        text: '',
        userId: 1
      }
    }
  },
  computed: {
    ...mapState(useNotesStore, ['notes'])
  },
  methods: {
    ...mapActions(useNotesStore, ['loadNotes', 'createNote']),
    async createCard() {
      await this.createNote(this.form)
      await this.loadNotes()
      this.$refs.noteForm.reset()
    }
  }
}

```

```

    },
    mounted() {
      this.loadNotes()
    }
  }
}
</script>

```

src/components/NoteCard.vue (Компонент карточки)

```

<template>
  <div class="card">
    <div class="card-body">
      <h5 class="card-title">{{ name }}</h5>
      <p class="card-text">{{ text }}</p>
    </div>
  </div>
</template>

<script>
export default {
  name: 'NoteCard',
  props: {
    name: { type: String, required: true },
    text: { type: String, required: false }
  }
}
</script>

```

src/stores/notes.js (Стейт-менеджер)

```

import { defineStore } from 'pinia'
import { notesApi } from '@/api'

const useNotesStore = defineStore('notes', {
  state: () => ({
    notes: []
  }),
  actions: {
    async loadNotes() {

```

```
        const response = await notesApi.getAll()
        this.notes = response.data
        return response
      },
      async createNote(data) {
        const response = await notesApi.createNote(data)
        this.notes = response.data
        return response
      }
    }
  })
})

export default useNotesStore
```

4. Вывод

В ходе выполнения лабораторной работы я на практике ознакомился с принципами работы пакетного менеджера NPM, который позволяет удобно управлять зависимостями проекта. Также были изучены базовые концепции фреймворка Vue.js: компонентный подход, система реактивности, хуки жизненного цикла mounted и работа с props.

Написанное приложение успешно выполняет свою функцию: оно связывает пользовательский ввод с глобальным хранилищем Pinia, асинхронно обменивается данными с API посредством библиотеки Axios и корректно обновляет пользовательский интерфейс без перезагрузки страницы.